

PROJET GDAETF2

Aide à la décision

Choix d'une bibliothèque PDF

Comparaison des solutions utilisables avec CakePHP 5.3 pour préparer l'export PDF sécurisé des DAE prévu par l'ADR 0062.

Choix recommandé : mPDF

Document préparatoire — 6 octobre 2026

Table des matières

1	Objet et contexte	2
2	Critères de décision	2
3	Bibliothèques candidates	2
3.1	mPDF — mpdf/mpdf	2
3.2	Dompdf — dompdf/dompdf	3
3.3	TCPDF — tecnickcom/tcpdf	3
3.4	wkhtmltopdf — knplabs/knp-snappy	3
3.5	Browsershot — spatie/browsershot	3
3.6	WeasyPrint	3
3.7	FPDF — setasign/fpdf	4
4	Matrice de décision	5
5	Recommandation	6
6	Architecture d'intégration proposée	6
7	Plan de validation avant implantation	6
8	Conclusion	7

1 Objet et contexte

Ce document prépare le choix de la bibliothèque de génération PDF pour l'application CakePHP 5.3. Le dépôt impose PHP ≥ 8.2 , une génération côté serveur, des dépendances hébergées dans l'environnement de l'application et l'absence de conversion réalisée dans le navigateur.

L'[ADR 0062](#) prévoit notamment :

- une route protégée `/applicationforms/viewpdf/{id}`;
- un service dédié et un gabarit de présentation local;
- la réutilisation de la Policy et des `FieldAuthorizations`;
- un tableau du cycle de validation, avec les statuts Accepté, Refusé et En attente;
- un type MIME `application/pdf` et un nom de fichier explicite.

Le PDF ne doit donc pas constituer une seconde source de vérité pour les droits ni nécessiter de modification du schéma de données.

2 Critères de décision

Les critères ci-dessous sont pondérés selon le besoin de l'application.

Critère	Poids	Interprétation
Compatibilité PHP / CakePHP	20 %	Installation et utilisation cohérentes avec PHP 8.2 et CakePHP 5.3.
Fidélité HTML / CSS	20 %	Capacité à utiliser un gabarit local sans réécrire toute la mise en page.
Mise en page paginée	15 %	Sauts de page, en-têtes, pieds de page et tableaux multipages.
Tables et workflow	15 %	Rendu lisible du tableau des étapes et des votes.
Déploiement Docker	15 %	Absence de binaire ou de runtime externe à maintenir.
Performance	5 %	Temps de génération et consommation mémoire raisonnables.
Maintenance	5 %	Pérennité du projet et clarté de son écosystème.
Testabilité	5 %	Facilité de tester le contenu, la réponse HTTP et le rendu.

TAB. 1 : Critères et pondération.

La note de chaque solution est comprise entre 1 (défavorable) et 5 (excellent). Le score final est la moyenne pondérée des notes.

3 Bibliothèques candidates

3.1 mPDF — `mpdf/mpdf`

Bibliothèque PHP pure, conçue pour produire des documents paginés à partir d'un sous-ensemble HTML/CSS. Elle convient particulièrement aux rapports, aux tableaux, aux en-têtes et pieds de page, ainsi qu'aux documents comportant du contenu français et des polices configurées localement.

Forces : pagination métier, tableaux, en-têtes, pieds de page, intégration Composer et absence de dépendance système.

Limites : le support du CSS moderne n'est pas celui d'un navigateur et la consommation mémoire doit être mesurée sur les DAE les plus volumineuses.

3.2 Dompdf — [dompdf/dompdf](#)

Bibliothèque PHP pure avec une intégration rapide pour un gabarit HTML/CSS classique. Elle est adaptée à un document composé de blocs et de tableaux simples.

Forces : simplicité, déploiement léger, API accessible et bon choix pour un premier prototype.

Limites : support limité de Flexbox/Grid et contrôle moins prévisible des mises en page complexes ou des sauts de page.

3.3 TCPDF — [tecnickcom/tcpdf](#)

Bibliothèque PHP pure robuste, particulièrement adaptée lorsque le document est construit par programmation avec un contrôle précis des coordonnées et des éléments PDF.

Forces : contrôle bas niveau, stabilité et faible dépendance à HTML.

Limites : le gabarit HTML prévu par l'ADR 0062 serait moins naturel à exploiter et la mise en page demanderait davantage de code spécifique.

3.4 wkhtmltopdf — [knplabs/knp-snappy](#)

Solution fondée sur un binaire externe convertissant HTML/CSS via un moteur WebKit. Le paquet PHP sert d'adaptateur, mais le binaire doit être installé et maintenu dans l'image Docker.

Forces : très bon rendu du HTML/CSS classique et réutilisation de pages proches de l'affichage navigateur.

Limites : moteur WebKit ancien, installation système, gestion des processus et surface opérationnelle plus importantes.

3.5 Browsershot — [spatie/browsershot](#)

Solution pilotant Chromium via Node.js. Elle offre un rendu très proche d'un navigateur moderne.

Forces : excellent support CSS moderne et rendu fidèle.

Limites : ajout de Node.js et Chromium, image Docker plus lourde, temps de démarrage et exploitation plus complexes pour un document métier.

3.6 WeasyPrint

Moteur de composition PDF orienté documents imprimés, généralement utilisé via Python et un processus externe depuis PHP.

Forces : modèle paginé et CSS d'impression de qualité.

Limites : absence d'intégration PHP native et ajout d'une chaîne Python à l'exploitation CakePHP.

3.7 FPDF — **setasign/fpdf**

Bibliothèque légère de construction PDF programmatique.

Forces : simplicité, contrôle et faible empreinte.

Limites : aucune conversion HTML ; le positionnement, les retours à la ligne et la pagination devraient être développés dans le service.

4 Matrice de décision

Critère	mPDF	Dompdf	TCPDF	wkhtmltopdf	Browsershot	WeasyPrint
Compatibilité PHP / CakePHP	5	5	5	4	3	2
Fidélité HTML / CSS	4	3	2	5	5	5
Mise en page paginée	5	3	4	4	4	5
Tables et workflow	5	4	3	4	4	5
Déploiement Docker	5	5	5	2	1	1
Performance	3	4	4	3	2	3
Maintenance	4	4	4	3	4	4
Testabilité	4	4	4	3	3	3
Score pondéré / 5	4,55	3,95	3,65	3,65	3,25	3,65

TAB. 2 : Matrice de décision pondérée.

Solution	Lecture de la décision
mPDF	Meilleur compromis pour le besoin complet : documents paginés, tables, PHP pur et déploiement Docker simple.
Dompdf	Alternative raisonnable si le gabarit reste simple et si le prototype confirme la pagination.
TCPDF / FPDF	À réserver à un document entièrement programmatique, sans dépendre d'un gabarit HTML.
wkhtmltopdf / Browsershot / Weasy-Print	À réserver à un besoin fort de fidélité navigateur ou CSS d'impression justifiant des dépendances externes.

5 Recommandation

Recommandé de retenir `mpdf/mpdf` pour l'implantation, sous réserve d'un prototype de rendu avec une DAE représentative. `dompdf/dompdf` doit rester l'alternative si le gabarit retenu n'utilise que du HTML/CSS simple.

mPDF respecte le mieux la combinaison des contraintes suivantes :

- bibliothèque installable via Composer et exécutable en PHP pur ;
- absence de binaire ou de runtime externe dans le conteneur ;
- bonne prise en charge des tableaux et de la pagination ;
- séparation claire entre le service de collecte sécurisée et le gabarit ;
- tests possibles sans navigateur ni service distant.

Le choix ne dispense pas de vérifier les versions disponibles au moment de l'ajout dans Composer, ni de mesurer la consommation mémoire sur les documents les plus longs.

6 Architecture d'intégration proposée

La bibliothèque ne doit pas être appelée directement par le contrôleur au-delà de l'orchestration HTTP. L'architecture cible est :

1. `ApplicationformsController`
: `viewpdf()` charge la DAE et applique `ApplicationformPolicy` avant tout rendu ;
2. un service dédié collecte les données visibles pour l'identité courante, en réutilisant les autorisations existantes ;
3. un gabarit local prépare le contenu HTML du document ;
4. un adaptateur PDF encapsule mPDF et retourne le contenu binaire ;
5. la réponse CakePHP définit `application/pdf`, le nom du fichier et l'ouverture dans un nouvel onglet côté interface.

Aucune table, vue SQL ou migration n'est nécessaire pour produire le PDF.

7 Plan de validation avant implantation

1. Ajouter un prototype avec une DAE comportant toutes les zones visibles.
2. Vérifier les champs autorisés et l'absence réelle des champs interdits.
3. Vérifier les trois statuts du workflow et les commentaires autorisés.
4. Vérifier les sauts de page, l'en-tête, le pied de page et le tableau multipage.
5. Vérifier les accents, les polices, les montants et les dates en français.
6. Mesurer le temps de génération et la mémoire sur une DAE volumineuse.
7. Ajouter les tests unitaires du service et les tests HTTP de la réponse PDF.
8. Documenter le choix définitif dans l'ADR approprié avant d'ajouter la dépendance Composer.

8 Conclusion

Pour la première version, mPDF offre le meilleur équilibre entre qualité de document métier, intégration PHP, déploiement et testabilité. Dompdf constitue une alternative valable si le prototype montre que les exigences de pagination restent simples. Les moteurs fondés sur un navigateur ou un processus externe ne sont justifiés que par un besoin de rendu qui ne pourrait pas être couvert par une bibliothèque PHP pure.